



HASHDIT

HashDit

Smart Contract Security

ListaDao Idle Market Support Audit Report

Security Audit Report

ListaDao

May 21 2026 - May 26 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	21 May 2026 – 26 May 2026
Overall Risk [Optional]	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/179
Commit	feature/idle-market

Smart Contract List

No.	Contract Name	Verdict	Details
1	IdleOracle.sol	Informational	[I01]
			[I02]
			[I04]
			[I06]
2	IdleCollateralToken.sol	Informational	[I02]
			[I03]
			[I04]
			[I05]
			[I06]

Findings

[I01] IdleOracle.peek delegates non-idle assets to the resilient oracle

Contract IdleOracle.sol

Severity **Informational**

Description	Description The IdleOracle.peek function forwards any asset that is not the idle collateral directly to the RESILIENT_ORACLE. As a result, a misconfigured market (for example, one created with oracle = IdleOracle and collateralToken = real-token) would silently route through the resilient oracle, making IdleOracle behave as a transparent wrapper. <pre>JavaScript function peek(address asset) external view returns (uint256) { if (asset == IDLE_COLLATERAL) return 0; return IOracle(RESILIENT_ORACLE).peek(asset); }</pre>
	While the resulting pricing behavior is identical to the resilient oracle, this silent fall through can mask operator configuration mistakes. The risk is low because createMarket is gated by the OPERATOR role.
Recommendation	Consider documenting clearly that delegation to RESILIENT_ORACLE for non-idle assets (e.g. for loan token) is intentional behavior and ensure that the OPERATOR do not assign the IdleOracle for new market creations outside of Idle Markets
Status	Waiting for confirmation from the development team.

[I02] Missing test coverage for idle market invariants

Contract IdleMarketFork.t.sol

Severity **Informational**

Description	Description Moolah The current PR test suite only verifies that supplyCollateral reverts on the idle market, but it does not cover the other entry points (borrow, withdrawCollateral, liquidate, and repay) that are also expected to revert when called against the idle market.
	MoolahVault

Similarly, The current PR test suite at IdleMarketFork.t.sol exercises MoolahVault.deposit, MoolahVault.withdraw, and MoolahVault.reallocate against a vault configured with an idle market in both the supply queue and the withdraw queue. However, the share denominated entry points MoolahVault.mint and MoolahVault.redeem are never tested. While these functions reuse the same internal function calls (_supplyMoolah and _withdrawMoolah) as their asset denominated counterparts, they differ at the conversion layer: mint computes assets from shares with Ceil rounding, and redeem computes assets from shares with Floor rounding

Without these explicit assertions, the safety properties of the idle market are not locked in by tests. A future change to Moolah (such as reordering checks in borrow) or MoolahVault could silently weaken the idle market invariants without being detected by the existing test coverage.

Recommendation Consider extending the test suite with unit or fork tests that assert each of the remaining operations reverts on the idle market.

Status Waiting for confirmation from the development team.

[I03] Redundant mint function in IdleCollateralToken

Contract IdleCollateralToken.sol

Severity **Informational**

Description

The IdleCollateralToken.mint() function exists solely to revert unconditionally, signaling that the token is non-mintable. However, mint is not part of the IERC20 interface, so the only way to reach it is through an explicit cast to the concrete contract type or by raw selector, neither of which is a realistic invocation path.

Description

```
JavaScript
function mint(address, uint256) external pure {
    revert IdleCollateralNonMintable();
}
```

Neither Moolah nor MoolahVault calls this function, making it redundant in the same way as the burn function. It contributes bytecode and expands the public interface surface without providing any functional benefit.

Recommendation Consider removing the mint function from IdleCollateralToken. If the intent is to symbolically communicate that the token is non-mintable, a comment in the contract's documentation is sufficient to convey that property.

Status Waiting for confirmation from the development team.

[I04] Risks to lenders from unrestricted reallocation into idle markets

Contract MoolahVault.sol

Severity Informational

Description

The MoolahVault.setCap() function allows the curator to assign any cap value to a market, including the idle market. When setCap() is configured with cap = uint184.max on the idle market (which is indicated within the IdleMarketFork.t.sol test files), the allocator and bot roles gain unrestricted ability to route vault assets into a market that, by design, generates no yield.

JavaScript

```
// 7) Configure vault caps + queues.  
vault.setCap(realMarket, type(uint184).max);  
vault.setCap(idleMarket, type(uint184).max);
```

Description

Because lenders share a single pooled share price and cannot opt out of specific market exposure, the following risks arise:

- Yield dilution can occur as the allocator can shift any proportion of vault assets into idle, and the effective APR for lenders decreases. With no on-chain limit on the idle ratio, a fully idle vault produces 0% APR for all lenders.
- A compromised allocator or bot key can drain all real markets into idle in a single transaction, reducing yield to zero until manual intervention occurs. While reallocate() enforces totalWithdrawn == totalSupplied so funds cannot be stolen, lenders can still lose meaningful yield during the time a compromised key remains active.

Recommendation Consider enforcing a meaningful upper bound on the idle market cap rather than allowing cap = uint184.max.

Status Waiting for confirmation from the development team.

[I05] Lenders have no economic incentive to supply directly into idle markets

Contract Moolah.sol & MoolahVault.sol

Severity Informational

Description

The idle market design does not offer any direct benefit to regular users who supply into it, since idle markets accrue no interest.

- If a user supplies directly into the idle market on Moolah, the deposit is effectively stranded because reallocation by the curator cannot move those

funds, as the user is not holding any MoolahVault shares and the vault does not own that liquidity. They would then have to perform direct withdrawal via the Moolah.withdraw function accordingly.

- If a user instead deposits into a MoolahVault whose supply queue includes the idle market, the curator can reallocate that liquidity to serve as exit liquidity for other users withdrawing from real markets, which exposes the user to a loss of opportunity cost without any yield in return.

As such, In practice, the idle market loan token supply via Moolah.supply or Moolah.deposit/mint only appears useful as a tool for ListaDao administrators to provide supplementary exit liquidity when needed, rather than as a yield bearing destination for end users.

Recommendation

Consider clearly documenting the intended role of the idle market so that users understand it is an operational primitive for ListaDao administrators rather than a yield bearing option

Additionally, consider calling Moolah.setWhitelist to enforce only MoolahVault actions initiation and not allow direct Moolah actions.

Status

Waiting for confirmation from the development team.

[I06] IdleCollateralToken is not fully ERC-20 compliant and can be optimized

Contract IdleCollateralToken.sol

Severity **Informational**

Description

Description

The IdleCollateralToken contract is declared as IERC20 and IERC20Metadata, but it does not fully conform to the ERC-20 specification. All view function requirements (name, symbol, decimals, totalSupply, balanceOf, allowance) are satisfied, however the contract breaks the spec in three places.

1. First, transfer, transferFrom, and approve revert unconditionally, including for zero value calls, while EIP-20 explicitly requires zero value transfers to behave as normal transfers and emit the corresponding events.
2. Second, the Transfer and Approval events are never emitted over the contract's lifetime. In the Moolah idle market context this non compliance is intentional, since the only consumer is Moolah itself, which invokes transferFrom and transfer through SafeTransferLib purely to block collateral movement, and no real flow needs the spec mandated behavior.

3. Third, The IERC20, IERC20Metadata declaration exists only to satisfy Moolah's type expectations, so the contract effectively functions as an "ERC-20 shaped sentinel" rather than a true ERC-20 token.

As such, given the IdleCollateralToken.sol is not expected to fully conform with the ERC20 spec, as a gas optimization, the Transfer and Approval event declarations inherited from the IERC20 interface can also be removed since they are never emitted by this contract.

Recommendation	As a gas optimization, the Transfer and Approval event declarations inherited from the IERC20 interface can also be removed since they are never emitted by this contract.
-----------------------	--

Status	Waiting for confirmation from the development team.
---------------	---